

IC526 Advanced Compilers

Lec 06: Lexers

Yoonseung Kim

DGIST EECS

Fall 2026

Overview: Text To AST

Lexical Analysis (by lexer)

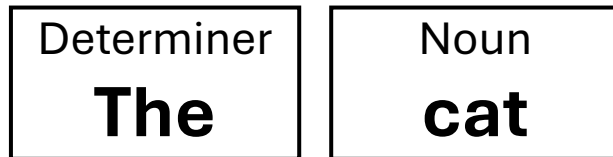
Syntax Analysis (by parser)

Analogy: Reading English Paragraph

Text:

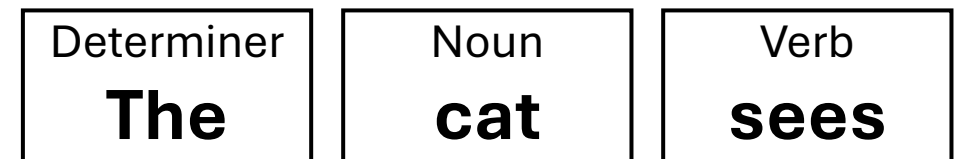
The cat sees a bird. A dog ate my food. She reads old books.

1. Lexical Analysis

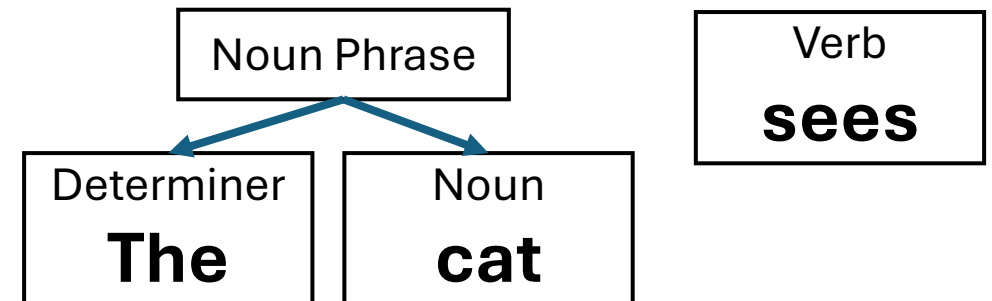


sees a bird. A dog ate my food. She reads old books.

2. Syntax Analysis

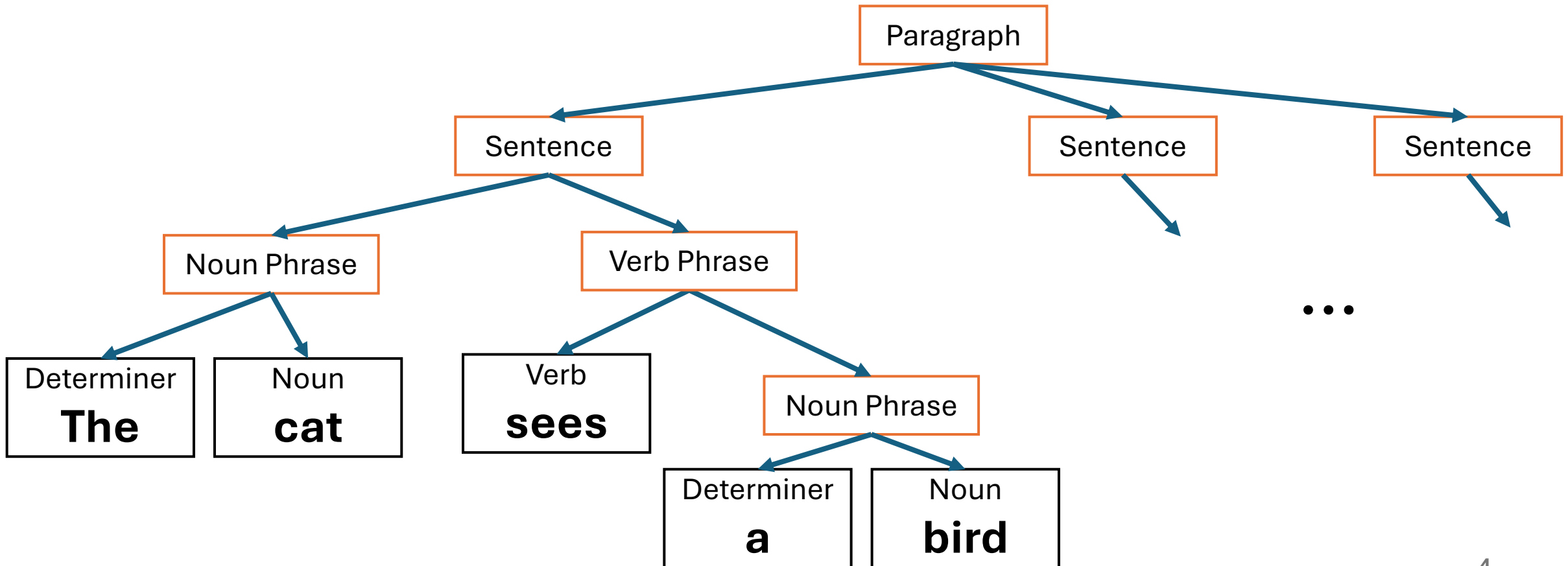


a bird. A dog ate my food. She reads old books.



Finally: A Derivation Tree

The cat sees a bird. A dog ate my food. She reads old books.



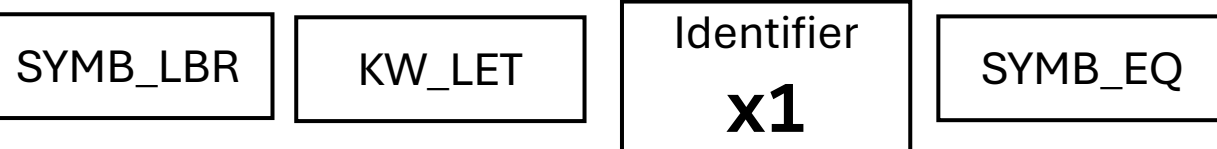
Reading Program Text

2. Syntax Analysis (by parser)

Text:

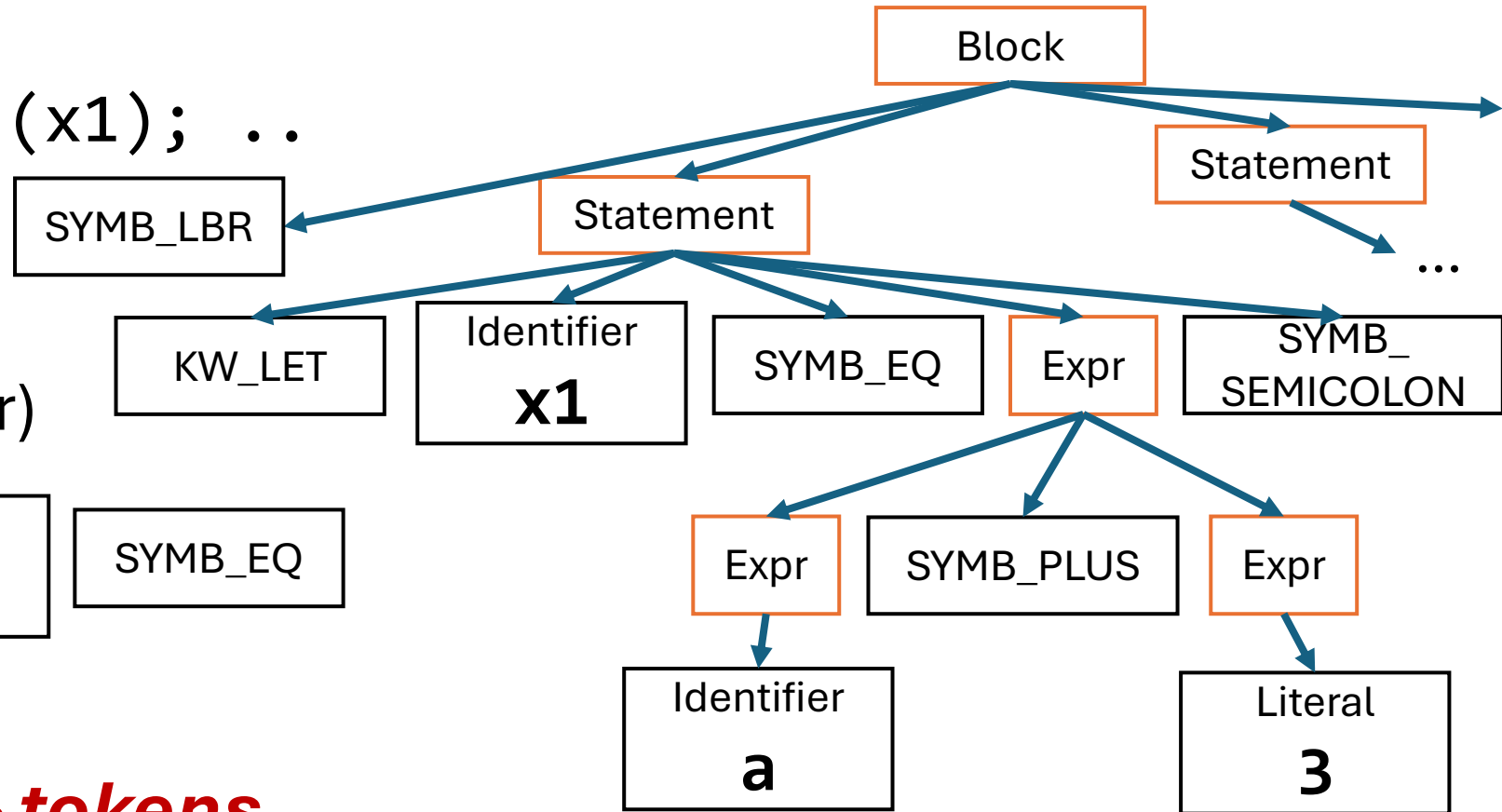
```
{ let x1 = a + 3; f (x1); ..
```

1. Lexical Analysis (by lexer)



```
a + 3; f (x1);
```

From left to right, you recognize ***tokens***.



Introduction to Lexers

Program is Given as Text

- A program is just a sequence of characters.

```
fn main() {  
    let x = 1 + 2;  
}
```

On disk, this is a flat sequence of characters written in the file:

'f' 'n' ' ' ' ' 'm' 'a' 'i' 'n' '(' ')' ' ' ' ' '{' '\n' '\t' '1' ... eof

Lexer: Characters To Tokens

- A lexer **groups** characters into meaningful units called **tokens**.

```
fn main() {  
    let x = 1 + 2;  
}
```

'f' 'n' ' ' 'm' 'a' 'i' 'n' '(' ')' ' ' ' ' '{' '\n' '\t' '1' ... eof

↓ Lexing (= Tokenizing, Scanning)

FN

ID("main")

(

)

{

LET

ID("x")

=

INT(1)

+

INT(2)

;

}

EOF

Keywords

Identifiers

Delimiters

Operators

Literals

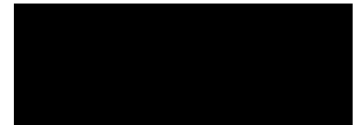
Specials

What Lexer Does

- Lexer reads the input file as a **stream**

Imagine multiple DFAs for each token kind
run simultaneously

'f'



FN

FALSE

ID()

What Lexer Does

- Lexer reads the input file as a **stream**

'f' 'n'



FN

FALSE

ID()

What Lexer Does

- Lexer reads the input file as a **stream**

With ‘ ’, no DFAs will survive

- (1) Go back to the last position that went to a final state of any token

'f' 'n' ' ' ' ' 

What Lexer Does

- Lexer reads the input file as a **stream**

With ‘ ’, no DFAs will survive

- (1) Go back to the last position that went to a final state of any token
- (2) Decide which token to generate.

In this case, we need a priority order.

- We want “fn” to be recognized as FN.

'f' 'n'

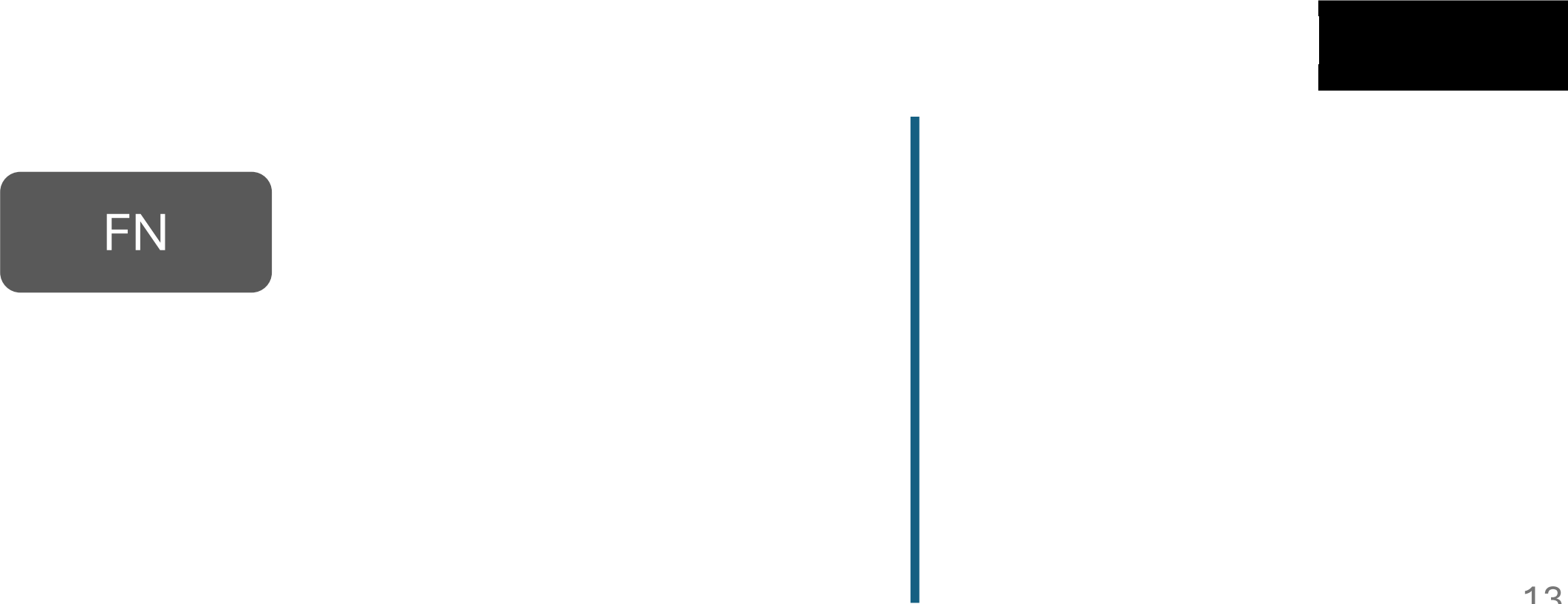


FN

ID()

What Lexer Does

- Lexer reads the input file as a **stream**



FN

What Lexer Does

- Lexer reads the input file as a **stream**

Whitespace ('\\n', '\\r', '\\t', ' ') between tokens
is usually skipped.
(exception: Python)

FN

' '



What Lexer Does

- Lexer reads the input file as a **stream**

FN

'm'



MUT

ID()

What Lexer Does

- Lexer reads the input file as a **stream**

A token may store a value.

FN

ID("main")

...

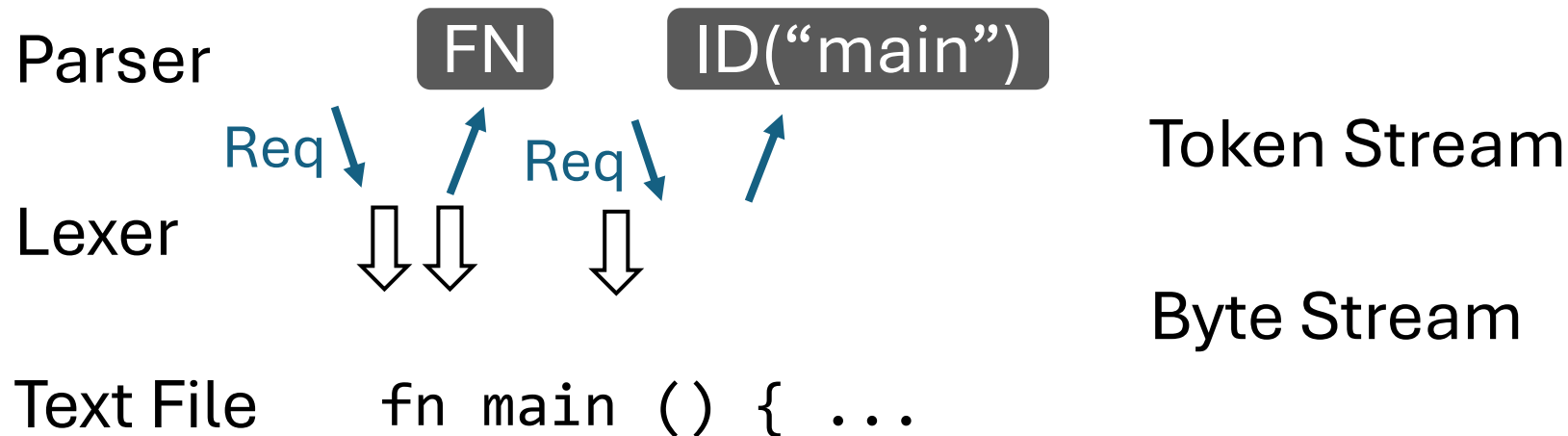
EOF

eof



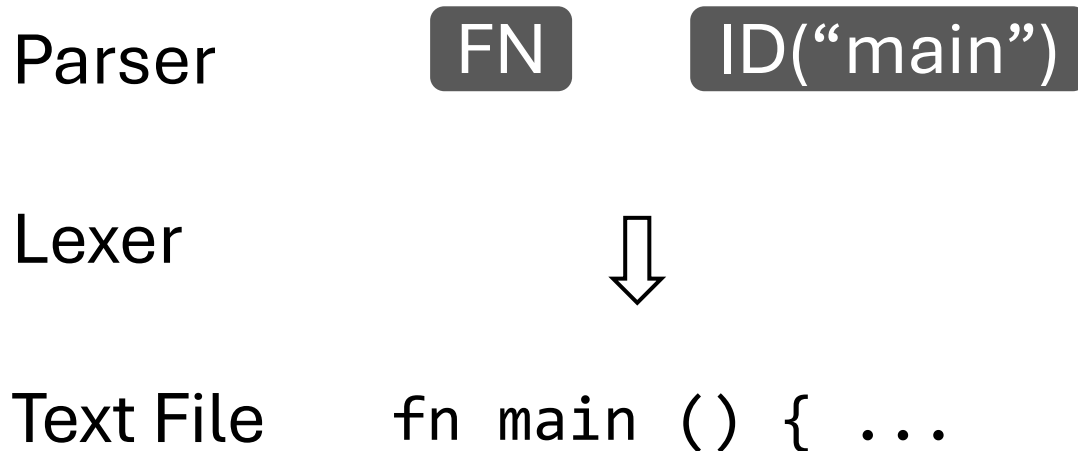
The Lexer–Parser Interface

- The parser calls the lexer whenever it needs the next token.
- The lexer reads from the current input position and returns one token.
 - The input position is saved for the next call.



More About Streams

- **Sequential access**
 - Consume elements in order: “Give me the next element.”
- **Allows on-demand production**
 - Produce the next element when requested.
 - No need to produce and store the entire sequence in advance.



How Lexers Are Designed

With Regular Expressions

Recall: Regular Expression (RE)

- Core constructs:
 - **Single character:** 'a' matches exactly the character 'a'
 - **Concatenation:** AB matches A followed by B
 - **Union:** A | B matches either A or B
 - **Repetition:** A* matches zero or more A's

c		{ 'c' }
x · y (or, xy)	Interpretation	{ 'xy' }
(x y)z		{ 'xz', 'yz' }
(xy)*		{ ϵ , 'xy', 'xyxy', ... }
		ϵ : empty string

Useful (Non-Standard) Notations of RE

- **Nonzero repetitions:** A^+ matches one or more A 's (i.e., $A^+ = AA^*$)
- **Roman alphabets:** $['A' - 'Z']$ for $'A' | 'B' | 'C' | \dots | 'Z'$ and $['a' - 'z']$ for ...
- **Digits:** $['0' - '9']$ for $'0' | '1' | \dots | '9'$
- **Combination of patterns:** $['A' - 'Z' _ '0' - '9']$ for $'A' | \dots | 'Z' | _ | '0' | \dots | '9'$
- **Exception:** $[^ 'A']$ for every byte except $'A'$
- Note: notations **do not improve the expressive power** of RE.

Now, Consider RustY Programs

- Can you express the set of strings for each token as regular expressions?

FN

LET

(

ID(_)

INT(_)

Some RE for RustY

$$\Sigma = \text{Bytes} \cup \{\text{eof}\} = \{a, \dots, 1, \dots, _ , (, \dots, '\backslash n', \dots, \text{eof}\}$$

FN 'fn'

LET 'let'

('('

ID(_) ?

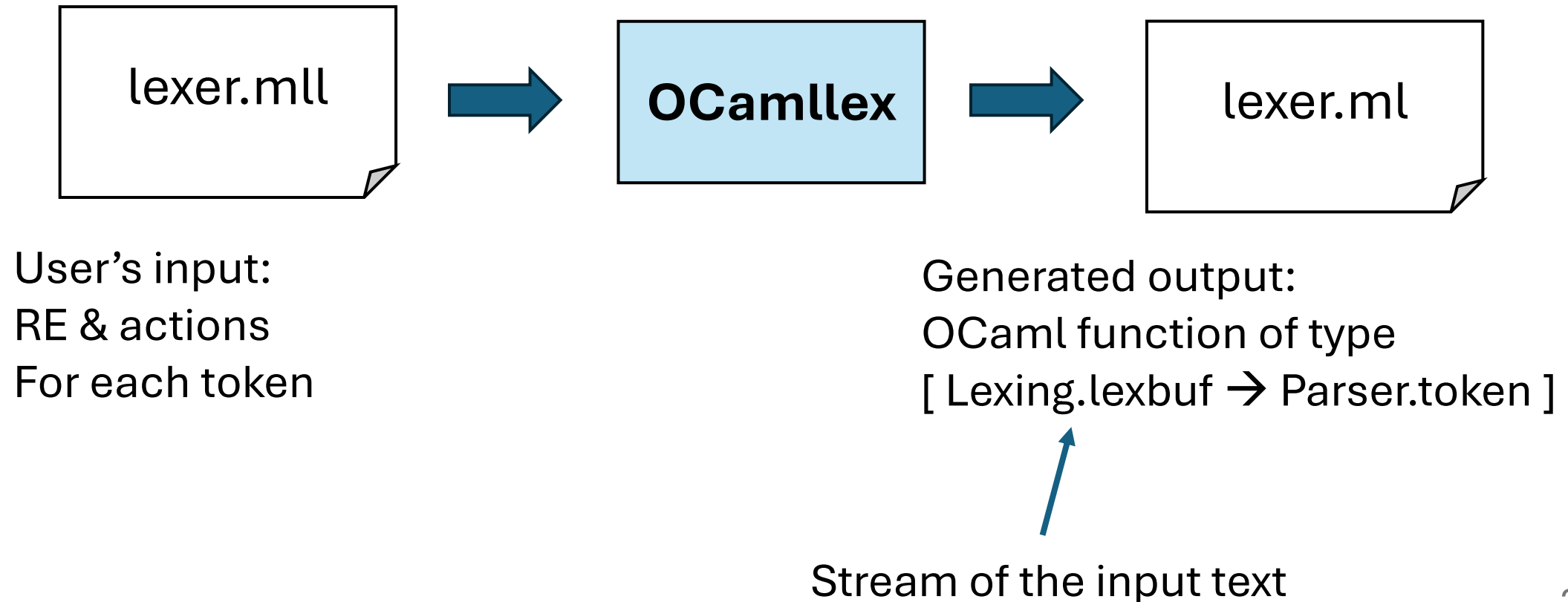
INT(_) ?

- Consists of **roman alphabets**, **underscores**, and **digits**
- Cannot start with a **digit**
- Alpha = ['A'-'Z' 'a'-'z' '_']
Digit = ['0'-'9']
Alpha(Alpha|Digit)*

- Consists of **digits**
- Digit+

ocamllex : A Lexer Generator

- ocamllex converts RE to DFA



Typical Structure of lexer.mll

```
{ (* 1. ocaml code *)
  open Parser (* Usually we define tokens in the parser module *)
  exception LexError of { msg: string; pos : Lexing.position }
}
```

} Header: OCaml code

```
(* 2. naming utility REs *)
let digit = ['0'-'9']
let alpha = ['a'-'z' 'A'-'Z' '_']
let ident = alpha (alpha | digit)*
let whitespace = [' ' '\t' '\r' '\n']
```

} RE definitions:
naming complex REs

```
(* 3. token generation rules *)
rule token = parse
  | whitespace { token lexbuf } (* recursive call *)
  | "fn" { FN }
  | "let" { LET }
  | ident as s { ID(s) }
  | ...
  | _ as c { raise (LexError { msg = "Lexer Error", .. }) }
```

} Rules: action for a
matched string

Header

```
{ (* OCaml Code *)  
  open Parser  
  exception LexError of { msg: string;  
                          pos: Lexing.position; }  
}
```

- With OCaml parser generators, the parser module usually defines the token type.
- The header may contain any OCaml code used by lexer actions:
 - E.g., Helper functions / exceptions.

Naming REs

```
let digit = ['0'-'9']  
let alpha = ['a'-'z' 'A'-'Z' '_']  
let ident = alpha (alpha | digit)*
```

- You can use the OCaml-like syntax:
[let <name> = <RE>]

Generation Rules

```
rule token = parse      (* rule <fname> = parse *)
  | "fn" { FN }         (* | <RE> { <Action> } *)
  | "let" { LET }
  | ...
```

- This will generate [let rec token (lexbuf: Lexing.lexbuf) = ..]
- Action: you decide what to return if a string matches an RE.
- The return type depends on the actions you assign.
 - ➔ Actions must return the identical type (= your token type).

Basic Cases

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ...
```

```
(* lexer.mll *)  
rule token = parse  
  | "fn" { FN }  
  | "let" { LET }  
  | ...
```

```
(* different namespace *)
```

Tokens with Payload

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ...
```

```
(* lexer.mll *)  
rule token = parse  
  | "fn" { FN }  
  | "let" { LET }  
  | ident as s { ID s } (* different namespace *)  
  | ... (* string inside ID token *)
```

Tokens with Payload

```
(* parser.ml *)
type token = FN | LET | ID of string | INT of int | ...

(* lexer.mll *)
rule token = parse                                (* different namespace *)
  | "fn" { FN }
  | "let" { LET }
  | ident as s { ID s }      (* string inside ID token *)
  | digit+ as s { INT (int_of_string s) }
  | ...
```

Ignoring Some Patterns

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ...
```

```
(* lexer.mll *)  
rule token = parse  
  | ...  
  | [' ' '\t' '\r' '\n'] { token lexbuf }  
  | "//" [^ '\n']* { token lexbuf }  
  | ...
```

Recursive call:
Get a token from the current stream



Any byte except
newline: '\n'



Ignoring Some Patterns

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ...
```

```
(* lexer.mll *)  
rule token = parse  
  | ...  
  | [' ' '\t' '\r'] { token lexbuf }  
  | '\n' { Lexing.new_line lexbuf; token lexbuf }  
  | "//" [^ '\n']* { token lexbuf }  
  | ...
```

Counts the line number:
for better error messages.



EOF and Exceptions

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ... | EOF
```

```
(* lexer.mll *)  
rule token = parse  
  | "fn" { FN }  
  | "let" { LET }  
  | ...  
  | eof { EOF }  
  | _ as c { raise (LexError {  
    msg = Printf.sprintf "unexpected char: %c" c;  
    pos = Lexing.lexeme_start_p lexbuf }) }
```

E.g.,
"let x = @"

Longest Match Policy

```
(* parser.ml *)  
type token = FN | LET | ID of string | INT of int | ...  
           | EQ | EQEQ | ...
```

We distinguish EQ(“=”) and EQEQ(“==”) since we don’t want to accept “1 == 3”.

```
(* lexer.mll *)  
rule token = parse  
  | “=” { EQ }  
  | “==” { EQEQ }  
  | ...
```

“1 == 3”

is converted to:

INT(1) EQEQ INT(3)

not

INT(1) EQ EQ INT(3)

Rule Priority: Breaking Ties

```
(* parser.m1 *)  
type token = FN | LET | ID of string | INT of int | ...
```

```
(* lexer.m1l *)  
rule token = parse  
  | "fn" { FN }  
  | "let" { LET }  
  | ident as s { ID s }  
  | ...
```

"fn"

FN



: Upper one wins.

ID("fn")

Usage of Lexer (From A String)

```
let source = "let fnx = 1 == 2; // note\n" in
let lexbuf = Lexing.from_string source in
let tkn1 = Lexer.token lexbuf in
let tkn2 = Lexer.token lexbuf in
[tkn1; tkn2] (* = [ LET ; ID("fnx")] *)
```

Assignment #2 Preview: `lexer.mll` (& `parser.mly`)

- You will be asked to write `[lexer.mll]` for our compiler `rustyc`